



Python – Einfache Datentypen

Stephan Karrer

Autor: Stephan Karrer

Ein Programmbeispiel

```
# Sample Comment

# Variablen
string1 = "Stephan"
string2 = 'Karrer'
numbers = range(1 , 10)

# Schleife
for i in numbers:
    # Anweisungen
    print("processing number:", i)

    # Fallunterscheidung
    if i % 2 == 0:
        print("even", i)
    else:
        print("odd", i)

# Anweisung nach der Schleife
print(string1, string2, "finished")
```

- Kommentar geht bis zum Ende der Zeile
- Jede Anweisung üblicherweise auf eigener Zeile (oder Komma-Separiert)
- Variablen ohne Typ-Angabe
- Groß/Klein-Schreibung ist relevant
- Bezeichner beginnen mit einem Buchstaben oder dem Unterstrich und dürfen neben diesen Zeichen auch Ziffern enthalten
- Einrückungen definieren die Anweisungsblöcke
- Köpfe von Schleifen und bedingten Anweisungen werden durch Doppelpunkt abgeschlossen

Variablen

```
a = 'Zeichenkette'    # str
print(a)
a = [1, 2, 3]          # list
print(len(a))

(a,b) = ('abc', 10)    # Tupel-Zuweisung
print(a,b)

(a,b) = (b, a)          # Werte-Tausch
print(a,b)

a = b = 'otto'         # Mehrfachzuweisung
print(a,b)
```

- Typ der Variablen ergibt sich zur Laufzeit anhand der Zuweisung
- Variablen sind nicht Typ-gebunden
- Natürlich muss der Typ zur Laufzeit zu den auszuführenden Operationen passen
- Mehrfachzuweisung und Tupel-Verwendung sind möglich

Ein/Ausgabe

```
eingabe = input("Bitte Eingabe: ")
print("Typ der Eingabe:", type(eingabe)) # <class 'str'>

zahl_als_string = input("Bitte Zahl: ")
zahl = int(zahl_als_string) # Cast
print("Typ von zahl", type(zahl)) # <class 'int'>
print(zahl - 1) # Subtraktion nur für Zahlen möglich
```

- Python bringt eine Vielzahl von eingebauten Funktionen mit
<https://docs.python.org/3/library/functions.html>
- Für einfache Ein- und Ausgaben sind wichtig:
 - `input` fordert den Benutzer zur Eingabe auf (Zeichenkette)
 - `print` gibt die Argumente durch Leerzeichen getrennt als Zeichenketten aus
 - `type` liefert zur Laufzeit den Typ
 - `int` macht aus Zeichenkette eine Ganzzahl (Objekt-Initialisierer), gibt es analog auch für andere Typen

Builtin-Datentypen

- Wie bei den Funktionen bringt Python auch einige eingebaute Standard-Datentypen mit <https://docs.python.org/3/library/stdtypes.html>
- Zu den wichtigsten gehören:
 - Wahrheitswerte (**bool**)
 - Zahlen (**int, float, complex**)
 - Zeichenketten (**str**)
 - Sequenzen (**list, tuple, range**)
 - Mengen (**set, frozenset**)
 - Maps (**dict**)
- Diese werden ergänzt durch Datentypen (Klassen) aus den Bibliotheken und potentiell durch uns programmierte Datentypen (Klassen)

Ganzzahlen

Bezeichner	int	
Literale	1, -2, 123456789123455 0o12, 0o677 0x12, 0x67FF 0b110011111, 0b1001	dezimal oktal hexadezimal binär
Beispiele	i = 1; a = 0xFF; k = 100_000 # Unterstrich wird durch Compiler ignoriert b = 0b1001_0001 # Führende 0 ist nicht erlaubt *=012	

- Wertebereich ist nur den vorhandenen Speicher begrenzt
- Unterstrich ist nur innerhalb des Ziffergebirges erlaubt
- Führende 0 ist nicht erlaubt

Gleitpunktzahlen

Bezeichner	float
Literale	1.1, 0.23, 123., 1., 0023.2 dezimal 1.0e3 # (represents 1.0×10³, or 1000.0) 1.166e-5 # (represents 1.166×10⁻⁵, or 0.00001166) 6.02214076e+23 # (represents 6.02214076×10²³, # or 602214076000000000000000.)
Beispiele	i = 96_485.332_123 # Unterstrich wird durch Compiler ignoriert j = 3.14_15_93

- Wertebereich ist abhängig von den zugrundeliegenden C-Types
- Unterstrich ist nur innerhalb des Ziffergebietes erlaubt
- Führende 0 ist erlaubt

Operationen mit Zahlen

Operation	Ergebnis
<code>- x</code>	Negation
<code>+ x</code>	x bleibt unverändert
<code>x + y</code>	Addition
<code>x - y</code>	Subtraktion
<code>x * y</code>	Produkt
<code>x / y</code>	Division – liefert float
<code>x // y</code>	Ganzzahldivision (Division mit Rest)
<code>x % y</code>	Rest bei der Division (Modulo)
<code>x ** n</code>	n-te Potenz von x
<code>pow(x,n)</code>	n-te Potenz von x
<code>abs(x)</code>	Absolutbetrag
<code>round(x [,n])</code>	Rundung
<code>int(x)</code>	Cast (Objektinitialisierung)
<code>float(x)</code>	Cast (Objektinitialisierung)

- Es gelten die üblichen Vorrangregeln, ansonsten Klammerung mit (...)
- Weitere stehen via Modul math zur Verfügung

Decimal als Datentyp

```
print(0.1 + 0.1 + 0.1 - 0.3)          # 5.551115123125783e-17
print(0.1 + 0.1+ 0.1+ 0.1+ 0.1+ 0.1+ 0.1+ 0.1+ 0.1)  #
0.9999999999999999
print(round(2.5))          # 2
print(round(3.5))          # 4

import decimal

print(decimal.getcontext())
decimal.getcontext().rounding=decimal.ROUND_DOWN
x = decimal.Decimal("0.1")
print(x + x + x - decimal.Decimal("0.3"))  # 0.0
print(round(decimal.Decimal("3.5")))        # 4: Context ignoriert
print(round(decimal.Decimal("3.5"), 0))      # 3: Context relevant
print(round(x+x+x+x+x))          # 0 - Rundung des Ergebnis
```

- Für kaufmännische Anwendungen ist float wenig geeignet!
- Python bringt mit Decimal aus der Bibliothek einen geeigneten Typ mit <https://docs.python.org/3/library/decimal.html#>

Zeichenketten

Bezeichner	str
Literale	'a', 'Das', "Hallo", """"Dies ist ein mehrzeiliger String""""
Operatoren	+ Konkatenation * Vervielfachung
Beispiele	s = "Hallo" s = 'Say "Hello", please.' char = 'B\u00E4h' # Bäh x = s + ' ' + char ("spam " "eggs") == "spam eggs" line = '*' * 50

- Zeichen können auch als Unicode-Sequenz (z.B. '\u00E4') eingegeben werden und es gibt weitere Möglichkeiten:
https://docs.python.org/3/reference/lexical_analysis.html#strings
- Vorsicht: In Ausdrücken werden einzelne literale Zeichenketten automatisch konkateniert

Escape-Squenzen

Escape Sequence	Meaning
<code>\<newline></code>	Ignored end of line
<code>\\</code>	Backslash
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code>\a</code>	ASCII Bell (BEL)
<code>\b</code>	ASCII Backspace (BS)
<code>\f</code>	ASCII Formfeed (FF)
<code>\n</code>	ASCII Linefeed (LF)
<code>\r</code>	ASCII Carriage Return (CR)
<code>\t</code>	ASCII Horizontal Tab (TAB)
<code>\v</code>	ASCII Vertical Tab (VT)
<code>\ooo</code>	Octal character
<code>\xhh</code>	Hexadecimal character
<code>\N{name}</code>	Named Unicode character
<code>\uxxxx</code>	Hexadecimal Unicode character
<code>\Uxxxxxxxx</code>	Hexadecimal Unicode character

Prefixed-Strings

```
# Raw String: verhindert die Interpretation des \  
r'\d{4}-\d{2}-\d{2}'  
R'\d{4}-\d{2}-\d{2}'  
  
# Strings mit Format-Spezifikationen  
who = 'nobody'  
nationality = 'Spanish'  
f'{who.title()} expects the {nationality} Inquisition!'
```

Strings als Sequenzen

```
word = 'Python'

# Zugriff via Index
word[0]          # character in position 0: P
word[5]          # character in position 5: n

word[-1]         # last character: n
word[-2]         # second-last character: o

# Slicing
word[0:2]        # characters from position 0 (included) to 2 (excluded)
word[2:5]        # characters from position 2 (included) to 5 (excluded)
word[:2]         # character from the beginning to position 2 (excluded)
word[4:]         # characters from position 4 (included) to the end
word[-2:]        # characters from the second-last (included) to the end
```

- Strings sind wie einige andere (Listen, Tupel, ...) Sequenz-Typen und weisen die allen Sequenzen gemeinsame Funktionalität auf

Methodik von Strings

```
wort = 'Python'  
wort[0] = 'B'
```

```
Liefert zur Laufzeit Fehler:  
Traceback (most recent call last):  
  File "E:\stephan\workspaces\pycharm\PythonProject\Zahlen.py",  
    line 74, in <module>  
      wort[0] = 'B'  
      ~~~~^^^  
TypeError: 'str' object does not support item assignment
```

- Strings können aber nicht direkt manipuliert werden (sind immutable)
- Verarbeitung geht nur über die Methodik der str-Klasse
<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

Gemeinsame Sequenz-Operationen

Operation	Result
<code>x in s</code>	<code>True</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>False</code>
<code>x not in s</code>	<code>False</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>True</code>
<code>s + t</code>	the concatenation of <code>s</code> and <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to adding <code>s</code> to itself <code>n</code> times
<code>s[i]</code>	<i>i</i> th item of <code>s</code> , origin 0
<code>s[i:j]</code>	slice of <code>s</code> from <i>i</i> to <i>j</i>
<code>s[i:j:k]</code>	slice of <code>s</code> from <i>i</i> to <i>j</i> with step <i>k</i>
<code>len(s)</code>	length of <code>s</code>
<code>min(s)</code>	smallest item of <code>s</code>
<code>max(s)</code>	largest item of <code>s</code>

Wahrheitswerte

Bezeichner	bool
Literale	True, False
Standardwert	False
Operatoren (absteigende Priorität)	not x Negation x and y Und mit partieller Auswertung x or y Oder mit partieller Auswertung Niedrigere Priorität als alle anderen Operatoren: <i>not x == y</i> entspricht <i>not (x == y)</i> und <i>x == not y</i> liefert Syntax-Fehler!

Vergleichsoperatoren

Operatoren	==	gleich
	!=	ungleich
	>	größer
	>=	größer oder gleich
	<	kleiner
	<=	kleiner oder gleich
	is	ist identisch
	is not	ist nicht identisch

- Haben alle gleiche Priorität
- Können verknüpft werden mit partieller Auswertung: $x < y <= z$
- Können nur verwendet werden, wo es Sinn macht
(Datentypen, sprich Klassen, können entsprechende Vergleichs-Methoden anbieten)

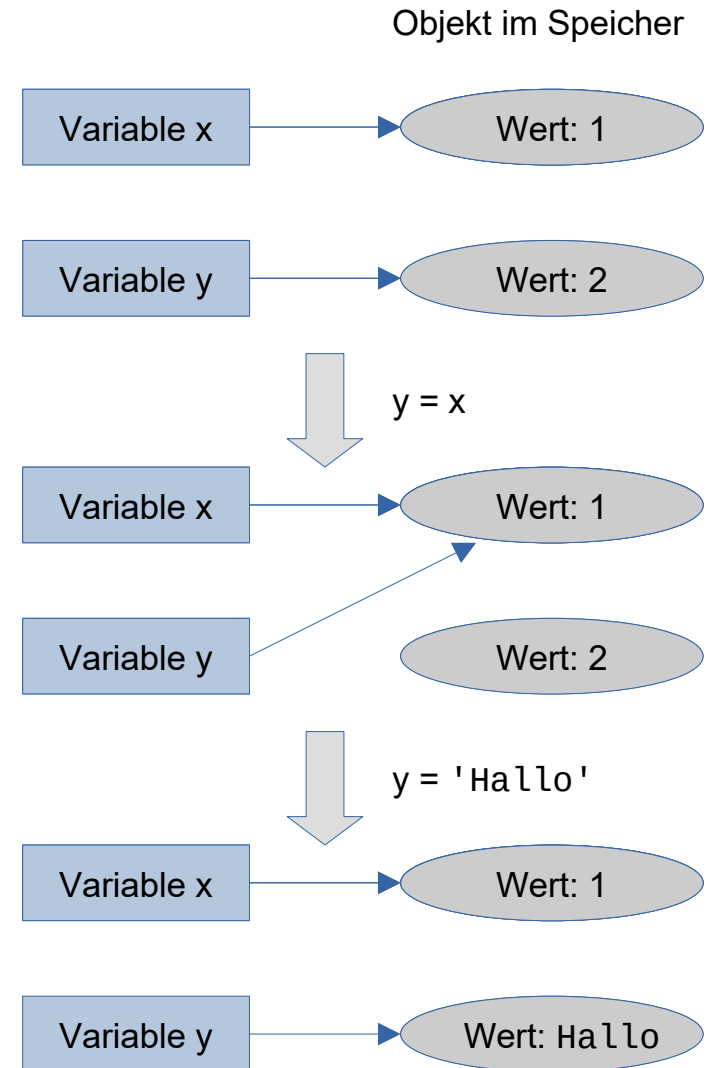
Variablen und Werte – Teil 1

```
x = 1
print(id(x))    # 140703929562024
print(id(1))    # 140703929562024
y = 2
print(id(y))    # 140703929562056
print(id(2))    # 140703929562056

y = x
print(id(x))    # 140703929562024
print(id(y))    # 140703929562024

y = 'Hallo'
print(x, y)     # 1 Hallo
print(id(x))    # 140703929562024
print(id(y))    # 2906441908896
```

- Zuweisung einer Variablen: Kopie der Referenz
- Dynamische Typisierung: Typkonvertierung (Cast) von Zahl zu Zeichenkette ist erstmal kein Problem



Variablen und Werte – Teil 2

- Python kopiert keine Werte (Objekte) im Speicher !
- Das nennt sich Referenz-Semantik
- Sofern ein Objekt bei komplexeren Szenarien nicht mehr benötigt wird, wird sich die automatische Speicherbereinigung kümmern (Garbage Collector)

```
x = 1
print(id(x))  # 140703931528104
y = x
print(id(y))  # 140703931528104
y = 2
print(id(y))  # 140703931528136
print(id(x))  # 140703931528104
z = 2
print(id(z))  # 140703931528136

x = [1, 2, 3]  # Liste
print(id(x))  # 2478157701952
y = x
y[1] = 4
print(id(y))  # 2478157701952
print(y)      # [1, 4, 3]
```

Variablen und Werte – Teil 3

```
x = 1
print(id(x)) # 140703931528104
z = y = x
print(id(y)) # 140703931528104
print(id(z)) # 140703931528104

del y, z    # löscht die Referenzen!
a = z       # Fehler!
```

```
Traceback (most recent call last):
  File "E:\pycharm\PythonProject\Sample.py", line 29, in <module>
    a = z
      ^
NameError: name 'z' is not defined
```

- Referenzen können durch den Entwickler gelöscht werden, Werte nicht!
- Sofern ein Objekt bei komplexeren Szenarien nicht mehr benötigt wird, wird sich die automatische Speicherbereinigung kümmern (Garbage Collector)

Referenz-Semantik: Das gleiche ist nicht unbedingt dasselbe

- `==` testet auf wertmäßige Gleichheit
- `is` testet, ob identisch

```
x = 1
y = 1.0

print(x == y)  # True
print(x is y)  # False

print(id(x))   # 140703777715112
print(id(y))   # 2703769907600
```